

Reviewer Pack — Minimal Order of Automorphism Groups and Tseitin Formula Com...

Entry 0d3096c1b61c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 08:21:40 UTC

Minimal Order of Automorphism Groups and Tseitin Formula Complexity

Entry ID: 0d3096c1b61c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 08:21:40 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (specifically, automorphism groups of finite groups)

Field B (complexity object): Tseitin Formulas

Statement:

For every d -regular graph G , the minimal order of the automorphism group $A(G)$ is linearly correlated with the size of its associated Tseitin formula φ_G , such that $\log|A(G)| = \Theta(|\varphi_G|)$.

Rationale (proposer's reasoning):

Automorphism groups capture symmetry in a structure and their order can be related to the complexity of expressing the graph's properties. Tseitin formulas are used to represent graphs and their size directly relates to computational complexity. If there is a strong correlation between these two measures, it could imply a deep connection between graph symmetry and computational difficulty.

Taxonomy category: GROUP_THEORY_TO_TSEITIN (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32972c3482075c01

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the Pearson correlation coefficient between $\log|A(G)|$ and $|\varphi_G|$ for all d -regular graphs G in $\{3, 4, \dots, 10\}$ is ≥ 0.8 , and for at least 80% of the seeds, the calculated values meet either ' $\log|A(G)| / |\varphi_G| \leq 1.5$ ' or ' $\log|A(G)| / |\varphi_G| \geq 0.5$ '. The conjecture is falsified if these conditions are not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	SAFE	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphism groups finite groups AND Tseitin formulas - minimal order automorphism group linearly correlated Tseitin formula complexity - Tseitin formula size relationship with automorphism group order

Top relevant hits considered: - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/q-alg/9712029v3] Quasi-Hopf twistors for elliptic quantum groups - [http://arxiv.org/abs/2104.14760v4] Automorphism and outer automorphism groups of Right-angled Artin groups are not relatively hyperbolic - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/math/0608534v2] On Automorphisms of Finite p -groups - [http://arxiv.org/abs/1509.06670v2] Automorphism Groups and Invariant Theory on PN - [http://arxiv.org/abs/1807.00547v3] Realisation of groups as automorphism groups in categories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(d, n):
        if d * (n - 1) % 2 != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d and (i, j) not in edges_added:
                    graph[i].append(j)
                    graph[j].append(i)
                    edges_added.add((i, j))
        return graph

    def automorphism_group_size(graph):
        n = len(graph)
        visited = [False] * n
        group_size = 1
```

```

def dfs(node, perm):
    if visited[node]:
        return True
    visited[node] = True
    for neighbor in graph[node]:
        if not visited[neighbor]:
            new_perm = perm[:]
            new_perm[neighbor] = perm[node]
            if not dfs(neighbor, new_perm):
                return False
    return True

def find_fixed_points(perm):
    fixed_points = []
    for i in range(n):
        if perm[i] == i:
            fixed_points.append(i)
    return fixed_points

def generate_permutations(fixed_points):
    n_fixed = len(fixed_points)
    if n_fixed == 0:
        yield {}
        return
    for p in itertools.permutations(range(n_fixed)):
        perm = {fixed_points[i]: fixed_points[p[i]] for i in range(n_fixed)}
        yield perm

def is_valid_permutation(perm):
    for node in range(n):
        if perm[node] != node:
            continue
        new_graph = {i: [j for j in graph[i] if j not in perm.values()] for i in range(n)}
        if not dfs(node, perm):
            return False
    return True

fixed_points = find_fixed_points({})
for perm in generate_permutations(fixed_points):
    if is_valid_permutation(perm):
        group_size *= len(perm)

return group_size

def tseitin_formula(graph):
    n = len(graph)
    clauses = []
    for i in range(n):
        clause = [f"X{i}"]
        for j in graph[i]:
            clause.append(f"-X{j}")
        clauses.append(clause)
    for i in range(n):
        for j in range(i + 1, n):
            if j not in graph[i]:
                clause = [f"-X{i}", f"-X{j}", f"X{2 * n + i}"]

```

```

        clauses.append(clause)
        clause = [f"-X{i}", f"-X{j}", f"X{2 * n + j}"]
        clauses.append(clause)
    for i in range(n):
        for j in range(i + 1, n):
            if j not in graph[i]:
                clause = [f"-X{2 * n + i}", f"-X{2 * n + j}", f"X{i}"]
                clauses.append(clause)
                clause = [f"-X{2 * n + i}", f"-X{2 * n + j}", f"X{j}"]
                clauses.append(clause)
    return clauses

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(n)) / n)
    return cov_xy / (std_x * std_y)

d_values = [3, 4, 5, 6, 7, 8, 9, 10]
results = []
for d in d_values:
    for _ in range(30):
        n = random.randint(5, 40)
        graph = generate_d_regular_graph(d, n)
        if graph is None:
            continue
        group_size = automorphism_group_size(graph)
        formula = tseitin_formula(graph)
        results.append((math.log(group_size), len(formula)))

if not results:
    return {
        "metric_name": "Pearson Correlation",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

x, y = zip(*results)
correlation = pearson_correlation(x, y)
mean_C = sum(math.log(group_size) / len(formula) for group_size, formula in results) / len(results)
support_fraction = sum(1 for _, _ in results if 0.5 <= math.log(group_size) / len(formula) <= 0.8) / len(results)

return {
    "metric_name": "Pearson Correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(len(formula) for _, formula in results),
    "conjecture_holds": correlation >= 0.8 and support_fraction >= 0.8,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    correlation_values = [r["metric_value"] for r in results if "metric_value" in r and not math.isnan(r["metric_value"])]
    support_fractions = [r["conjecture_holds"] for r in results]

    mean_corr = sum(correlation_values) / len(correlation_values)
    std_corr = math.sqrt(sum((x - mean_corr) ** 2 for x in correlation_values) / len(correlation_values))
    support_fraction = sum(support_fractions) / len(support_fractions)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{first_failing_seed}\"")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_53f229d2.py", line 158, in <module>
    trial_result = run_trial(seed)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_53f229d2.py", line 124, in run_trial
    group_size = automorphism_group_size(graph)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_53f229d2.py", line 76, in automorphism_group_size
    fixed_points = find_fixed_points({})
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_53f229d2.py", line 54, in find_fixed_points
    if perm[i] == i:
        ~~~~^
KeyError: 0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient and support fraction. | next: Re-run the test with appropriate error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23418
2	propose	ollama_remote	glm4:latest	0	0	17200
3	propose	ollama_remote	glm4:latest	0	0	16826
4	preregistration	ollama_remote	glm4:latest	0	0	17045
5	novelty	ollama_remote	glm4:latest	0	0	17749
6	novelty	ollama_remote	glm4:latest	0	0	12137
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16593
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23967
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14568
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25347
11	judge	ollama_remote	glm4:latest	0	0	8758

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193608 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/0d3096c1b61c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0d3096c1b61c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0d3096c1b61c.tar.gz (if generated)